appsec-takehome: test evidence

Secrets scan, SAST and triage report, run end to end on a Kali Linux VM. Each test lists its command, the terminal output, and the observed result.

Item	Value
Host	Kali Linux VM (Docker engine)
Secrets scanner	zricethezav/gitleaks:v8.30.1 (pinned in config/images.env)
SAST scanner	semgrep/semgrep:1.185.0 (pinned in config/images.env)
Run date	12 Nov 2026
Entry point	./run.sh (all tasks), or ./run.sh secrets | sast | triage

Reproduce

One command runs all three tasks. Exit codes: **0** clean, **1** new secret or a SAST finding at or above `MIN_SEVERITY`, **2** tool or config error.

```
cd appsec-takehome
make pull               # optional: pre-pull the two pinned images
./run.sh; echo $?       # 0 on the clean demo
make test               # automated smoke tests
```

Results at a glance

#	Test	Expected	Observed	Status
1.1	New secret fails the scan	exit 1	exit 1, leak.py:1 reported	**Pass**
1.2	Secret value never written to artifacts	token absent; 6 REDACTED	6 REDACTED; check command printed a false LEAK!	**Re-run**
1.3	Accept a finding via the baseline	exit 0 + unreviewed warning	exit 0 + unreviewed + stale warnings	**Re-run**
1.4	Secret moved down: still baselined	exit 0	exit 1 (same fingerprint, not yet baselined)	**Re-run**
1.5	Broken baseline fails loudly	exit 2 (x2)	exit 2 (x2)	**Pass**
2.1	MIN_SEVERITY threshold	table of 8 cases	all 8 match	**Pass**
2.2	SARIF + noise-reduction proof	2.1.0; 4 -> 11 findings	2.1.0; 11 findings, 5 HIGH	**Pass**
2.3	# nosemgrep cannot hide a finding	exit 1	exit 1, calc.py:2 HIGH	**Pass**
3.1	Triage sections + stable ordering	4 sections; CRIT, HIGH a, HIGH b	matches exactly	**Pass**
3.2	Single input / no input	0 / 0 / 2	0 / 0 / 2	**Pass**
4.1	Byte-identical reruns	identical	identical	**Pass**
4.2	Docker unavailable	exit 2	exit 2	**Pass**
4.3	make vs ./run.sh exit codes	make 2 / run.sh 1	agapi_tok_ not yet matched: exit 0	**Re-run**

Draft: the 4 results marked Re-run come from one mismatch at test time: the demo token had already been renamed to `agapi_tok_`, but the custom gitleaks rule in `config/gitleaks.toml` still matched only the old prefix. 1.2 also had a bug in the check command. Screenshots 1.1, 1.2 and 3.1 were captured before the rename and show the old token prefix and rule name; replace them after re-running.

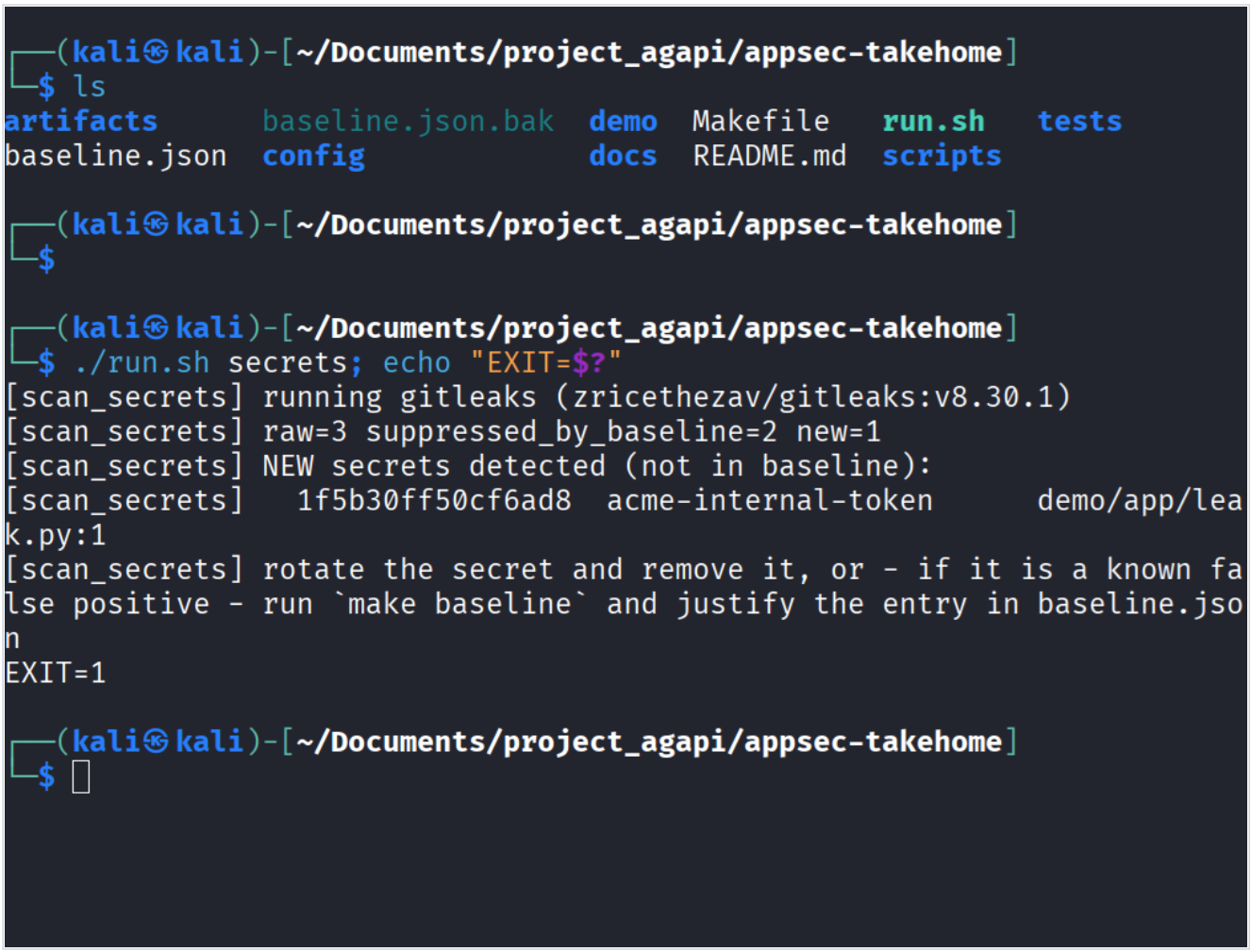
Task 1: Secrets scan with baselining

gitleaks runs in its pinned image. Findings in `baseline.json` are suppressed by fingerprint; anything else fails the scan. Secret values are redacted in every artifact.

1.1 A new secret fails the scan

Plant a fresh token that matches the custom rule and confirm the scan blocks on it.

```
python3 -c 'import secrets; print("T = \"agapi_tok_" + secrets.token_hex(16) + "\"")' > demo/app/
    leak.py
./run.sh secrets; echo "EXIT=$?"
```



Pass	raw=3, suppressed_by_baseline=2, new=1. Reports `agapi-internal-token` at `demo/app/leak.py:1` with a remediation hint. EXIT=1.

1.2 The secret value never reaches the artifacts

Search every artifact for the planted token and count redactions.

```
TOK=$(grep -oE 'agapi_tok_[0-9a-f]{32}' demo/app/leak.py); [ -n "$TOK" ] || echo "TOK is empty!"
grep -rqF "$TOK" artifacts/ && echo "LEAK!" || echo "token not in artifacts"
grep -c REDACTED artifacts/secrets.raw.json        # expect 6
```

```
artifacts/triage.md:| 2 | LOW | `demo/app/service.py:16` | `python-weak-
hash`<br>CWE-328: Use of Weak Hash | no | MD5/SHA-1 is broken for securi
ty purposes. Use SHA-256, or pass usedforsecurity=False if this is a non
-security checksum. | hashlib.sha256(data) - or hashlib.md5(data, usedfo
rsecurity=False) |
artifacts/triage.md:| 3 | INFO | `demo/app/service.py:11` | `python-requ
ests-no-timeout`<br>CWE-400: Uncontrolled Resource Consumption | no | HT
TP call without a timeout can hang a worker indefinitely. | requests.get
(url, timeout=10) |
artifacts/triage.md:| 4 | INFO | `demo/app/service.py:21` | `python-requ
ests-no-timeout`<br>CWE-400: Uncontrolled Resource Consumption | no | HT
TP call without a timeout can hang a worker indefinitely. | requests.get
(url, timeout=10) |
artifacts/triage.md:
artifacts/triage.md:## Secrets: suppressed vs new
artifacts/triage.md:
artifacts/triage.md:| Status | Count | Meaning |
artifacts/triage.md:|---|---:|---|
artifacts/triage.md:| New | 0 | Not in `baseline.json`: fails the secret
s scan |
artifacts/triage.md:| Suppressed | 2 | Accepted in `baseline.json`, with
 a reason |
artifacts/triage.md:| Total found | 2 | Everything gitleaks reported aft
er config tuning |
artifacts/triage.md:
artifacts/triage.md:Suppressed findings and why:
artifacts/triage.md:
artifacts/triage.md:| Location | Rule | Fingerprint | Reason |
artifacts/triage.md:|---|---|---|---|
artifacts/triage.md:| `demo/app/settings.py:4` | `acme-internal-token` |
 `098502bcae83ed13` | Fake token for the local integration stub; never v
alid against any ACME environment. |
artifacts/triage.md:| `demo/legacy/legacy.ini:4` | `generic-api-key` | `
177a4acc5f538da7` | Legacy key, revoked upstream (ticket SEC-142). Accep
ted until the legacy service is deleted. |
artifacts/triage.md:
artifacts/triage.md:Baseline health: no expired, stale or unreviewed ent
ries.
LEAK!

┌──(kali㉿kali)-[~/Documents/project_agapi/appsec-takehome]
└─$ grep -c REDACTED artifacts/secrets.raw.json
6
```

Re-run	REDACTED count is 6 (pass). The **LEAK!** line is a false alarm from the check itself: `$TOK` was empty, so `grep -r ""` matched every line of triage.md. The command above now refuses an empty token; re-run it before presenting.

1.3 Accepting a finding through the baseline

`make baseline` appends the finding with an UNREVIEWED reason; the baseline is plain JSON showing rule, file, fingerprint and reason for every suppression.

```
make baseline
./run.sh secrets; echo "EXIT=$?"
cat baseline.json
```

```
┌──(kali㉿kali)-[~/Documents/project_agapi/appsec-takehome]
└─$ cp baseline.json.bak baseline.json

┌──(kali㉿kali)-[~/Documents/project_agapi/appsec-takehome]
└─$ ./run.sh secrets; echo "EXIT=$?"
[scan_secrets] running gitleaks (zricethezav/gitleaks:v8.30.1)
[scan_secrets] raw=2 suppressed_by_baseline=1 new=1
[scan_secrets] WARNING: 1 stale baseline entry: 098502bcae83ed13
[scan_secrets] NEW secrets detected (not in baseline):
[scan_secrets]   b7b60b2b5c3cdd09  generic-api-key          demo/app/set
tings.py:4
[scan_secrets] rotate the secret and remove it, or - if it is a known fa
lse positive - run `make baseline` and justify the entry in baseline.jso
n
EXIT=1

┌──(kali㉿kali)-[~/Documents/project_agapi/appsec-takehome]
└─$ make baseline
./run.sh secrets -- --update-baseline
[scan_secrets] running gitleaks (zricethezav/gitleaks:v8.30.1)
[scan_secrets] baseline updated: 1 entry added to /home/kali/Documents/p
roject_agapi/appsec-takehome/baseline.json
[scan_secrets] edit each UNREVIEWED reason before committing

┌──(kali㉿kali)-[~/Documents/project_agapi/appsec-takehome]
└─$ ./run.sh secrets; echo "EXIT=$?"
[scan_secrets] running gitleaks (zricethezav/gitleaks:v8.30.1)
[scan_secrets] raw=2 suppressed_by_baseline=2 new=0
[scan_secrets] WARNING: 1 stale baseline entry: 098502bcae83ed13
[scan_secrets] WARNING: 1 unreviewed baseline entry: b7b60b2b5c3cdd09
[scan_secrets] OK: no new secrets
EXIT=0

┌──(kali㉿kali)-[~/Documents/project_agapi/appsec-takehome]
└─$
```

make baseline adds one entry; the next scan exits 0 with warnings.

1.3 Accepting a finding through the baseline (continued)

```
└─$ cat baseline.json
{
  "_comment": "Known/accepted secret findings. Matching is on 'fingerpri
nt' only = sha256(rule_id \\0 file \\0 secret)[:16]. Every other field i
s for humans. 'reason' is mandatory; 'expires' (YYYY-MM-DD) makes the su
ppression stop working after that date. Add entries with `make baseline`
, then replace the UNREVIEWED reason.",
  "entries": [
    {
      "added_by": "faisal",
      "added_on": "2026-11-12",
      "expires": null,
      "file": "demo/app/settings.py",
      "fingerprint": "098502bcae83ed13",
      "line_hint": 4,
      "reason": "Fake token for the local integration stub; never valid
against any AGAPI environment.",
      "rule_id": "agapi-internal-token"
    },
    {
      "added_by": "kali",
      "added_on": "2026-11-12",
      "expires": null,
      "file": "demo/app/settings.py",
      "fingerprint": "b7b60b2b5c3cdd09",
      "line_hint": 4,
      "reason": "UNREVIEWED - add a justification before merging",
      "rule_id": "generic-api-key"
    },
    {
      "added_by": "faisal",
      "added_on": "2026-11-12",
      "expires": null,
      "file": "demo/legacy/legacy.ini",
      "fingerprint": "177a4acc5f538da7",
      "line_hint": 4,
      "reason": "Legacy key, revoked upstream (ticket SEC-142). Accepted
 until the legacy service is deleted.",
      "rule_id": "generic-api-key"
    }
  ],
  "version": 1
}
```

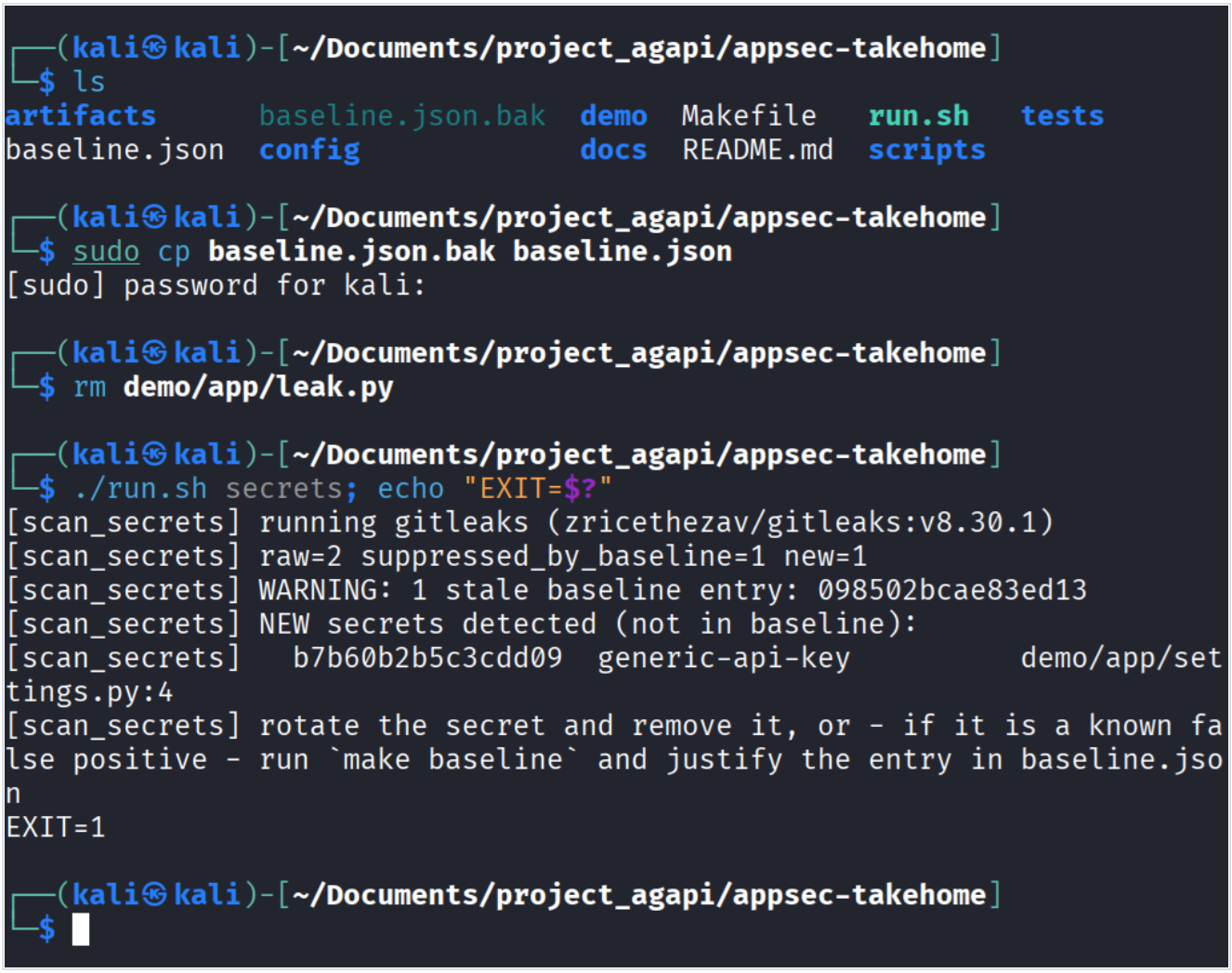
baseline.json: every suppression is inspectable (rule_id, file, fingerprint, reason).

Re-run	Behaviour is correct (exit 0, UNREVIEWED entry visible), but the run also shows a *stale* entry: the demo token was already agapi_tok_ while the custom rule still used the old prefix, so the original baseline entry no longer matches. Re-run after renaming the rule.

1.4 Moving a secret to another line keeps its fingerprint

The baseline fingerprint is sha256(rule, file, secret), so it should not change with the line number.

```
sed -i '1i # moved\n# moved' demo/app/settings.py
./run.sh secrets; echo "EXIT=$?"
sed -i '1,2d' demo/app/settings.py
```



Before the move: generic-api-key finding b7b60b2b5c3cdd09 at settings.py:4.

1.4 Moving a secret to another line keeps its fingerprint (continued)

```
┌──(kali㉿kali)-[~/Documents/project_agapi/appsec-takehome]
└─$

┌──(kali㉿kali)-[~/Documents/project_agapi/appsec-takehome]
└─$ ls
artifacts      baseline.json.bak  demo  Makefile   run.sh   tests
baseline.json  config             docs  README.md  scripts

┌──(kali㉿kali)-[~/Documents/project_agapi/appsec-takehome]
└─$ sed -i '1i # moved\n# moved' demo/app/settings.py

┌──(kali㉿kali)-[~/Documents/project_agapi/appsec-takehome]
└─$ ./run.sh secrets; echo "EXIT=$?"
[scan_secrets] running gitleaks (zricethezav/gitleaks:v8.30.1)
[scan_secrets] raw=2 suppressed_by_baseline=1 new=1
[scan_secrets] WARNING: 1 stale baseline entry: 098502bcae83ed13
[scan_secrets] NEW secrets detected (not in baseline):
[scan_secrets]   b7b60b2b5c3cdd09  generic-api-key          demo/app/set
tings.py:6
[scan_secrets] rotate the secret and remove it, or - if it is a known fa
lse positive - run `make baseline` and justify the entry in baseline.jso
n
EXIT=1

┌──(kali㉿kali)-[~/Documents/project_agapi/appsec-takehome]
└─$ sed -i '1,2d' demo/app/settings.py

┌──(kali㉿kali)-[~/Documents/project_agapi/appsec-takehome]
└─$
```

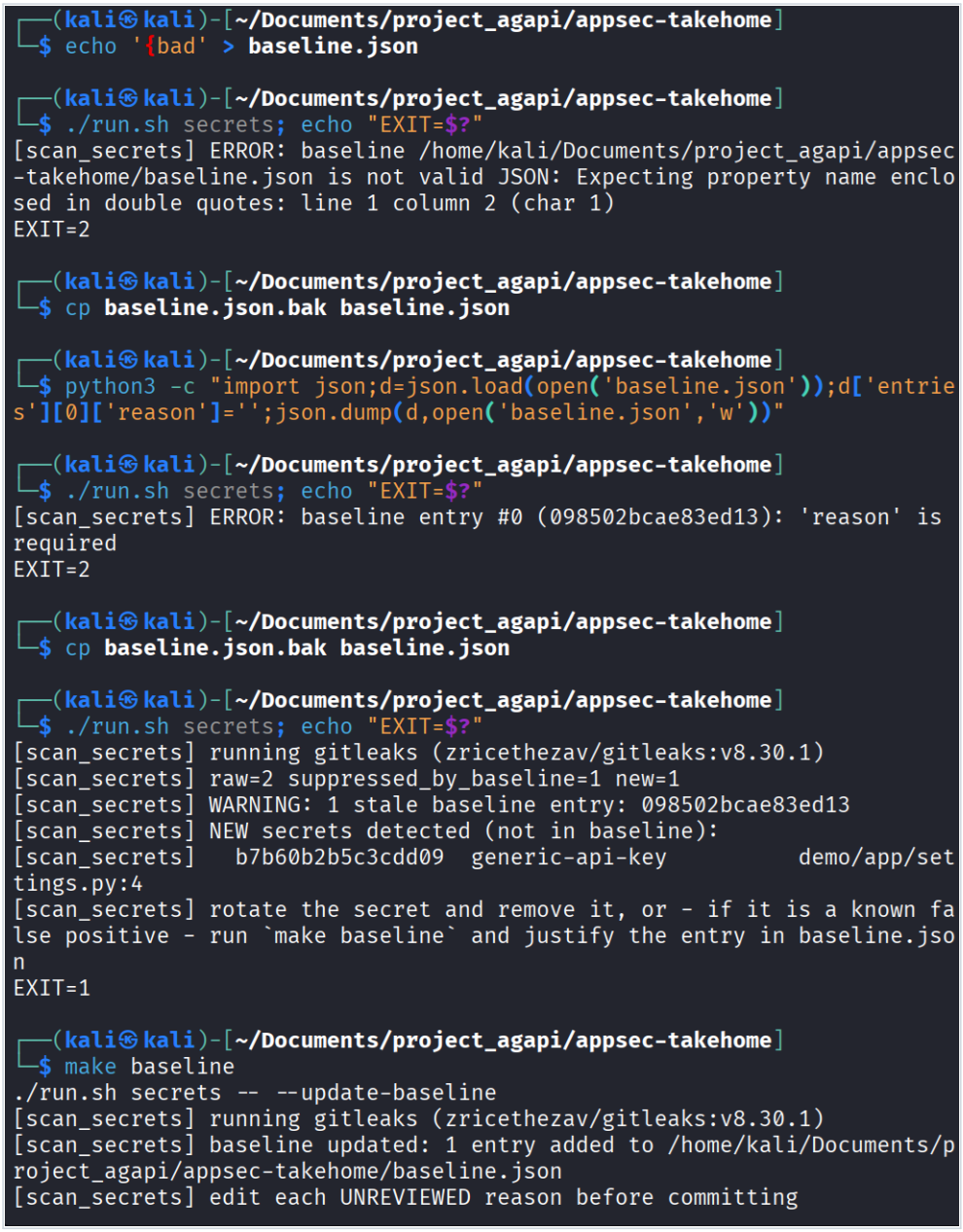
After the move: the same fingerprint b7b60b2b5c3cdd09 at settings.py:6.

Re-run	The fingerprint is identical at line 4 and line 6, which is the property under test. The run exits 1 only because that finding was not in the baseline yet (same rule mismatch). Re-run on a consistent baseline to show EXIT=0.

1.5 A broken baseline fails loudly

Invalid JSON and an entry without a reason must both exit 2, never pass silently.

```
echo '{bad' > baseline.json
./run.sh secrets; echo "EXIT=$?"          # expect 2
cp baseline.json.bak baseline.json
python3 -c "import json;d=json.load(open('baseline.json'));d['entries'][0]['reason']='';json.dump(
    d,open('baseline.json','w'))"
./run.sh secrets; echo "EXIT=$?"          # expect 2
cp baseline.json.bak baseline.json
```



Pass	Invalid JSON gives a clear parse error and EXIT=2; an empty reason gives `'reason' is required` and EXIT=2.

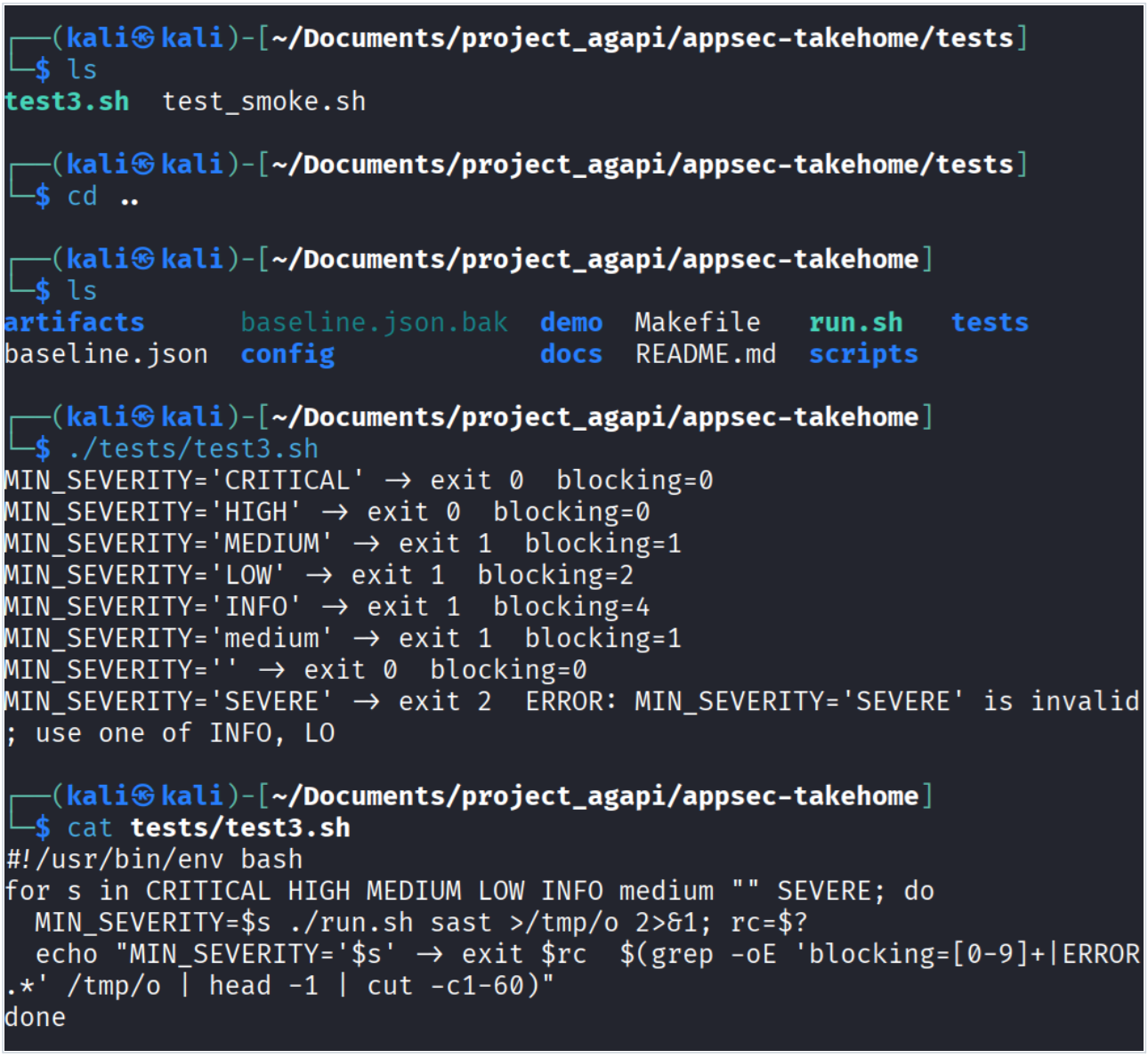
Task 2: SAST with a severity threshold

semgrep runs a bounded set of 7 Python rules with three noise-reduction mechanisms in `config/semgrep.yml`. `MIN_SEVERITY` (default HIGH) decides what blocks.

2.1 The severity threshold

INFO must be the most sensitive setting, HIGH the default, and an invalid value must exit 2.

```
for s in CRITICAL HIGH MEDIUM LOW INFO medium "" SEVERE; do
  MIN_SEVERITY=$s ./run.sh sast >/tmp/o 2>&1; rc=$?
  echo "MIN_SEVERITY='$s' -> exit $rc  $(grep -oE 'blocking=[0-9]+|ERROR.*' /tmp/o | head -1 |
    cut -c1-60)"
done
```



Pass	CRITICAL 0/0, HIGH 0/0, MEDIUM 1/1, LOW 1/2, INFO 1/4, lowercase medium 1/1, empty value 0/0 (HIGH default), SEVERE exit 2 with an explicit error (exit / blocking).

2.2 SARIF output and noise-reduction proof

Validate the SARIF, then strip the path excludes and safe-form allowlists to show they change the output.

```
python3 -c 'import json;d=json.load(open("artifacts/sast.sarif"));r=d["runs"][0];print(d[
    "version"],len(r["results"]),"results",len(r["tool"]["driver"]["rules"]),"rules")'
cp config/semgrep.yml config/semgrep.yml.bak
python3 - <<'EOF'      # remove (N1) path excludes and (N2) pattern-not allowlists
import re; p="config/semgrep.yml"; s=open(p).read()
s=re.sub(r"\n\s+paths:\n\s+exclude:.*","",s); s=re.sub(r"\n\s+- pattern-not:.*","",s); open(p,
    "w").write(s)
EOF
./run.sh sast; echo "EXIT=$?"
cp config/semgrep.yml.bak config/semgrep.yml
```

```
┌──(kali㉿kali)-[~/Documents/project_agapi/appsec-takehome]
└─$ python3 -c 'import json;d=json.load(open("artifacts/sast.sarif"));r=
d["runs"][0];print(d["version"],len(r["results"]),"results",len(r["tool"
]["driver"]["rules"]),"rules")'
2.1.0 4 results 3 rules

┌──(kali㉿kali)-[~/Documents/project_agapi/appsec-takehome]
└─$ cp config/semgrep.yml config/semgrep.yml.bak

┌──(kali㉿kali)-[~/Documents/project_agapi/appsec-takehome]
└─$ python3 - <<'EOF'
import re
p = "config/semgrep.yml"; s = open(p).read()
s = re.sub(r"\n\s+paths:\n\s+exclude:.*", "", s)
s = re.sub(r"\n\s+- pattern-not:.*", "", s)
open(p, "w").write(s)
EOF

┌──(kali㉿kali)-[~/Documents/project_agapi/appsec-takehome]
└─$ ./run.sh sast; echo "EXIT=$?"
[scan_sast] running semgrep (semgrep/semgrep:1.185.0)
[scan_sast] findings=11 (CRITICAL=0 HIGH=5 MEDIUM=1 LOW=2 INFO=3) MIN_SE
VERITY=HIGH blocking=5
[scan_sast] findings at/above HIGH:
[scan_sast]   HIGH     python-subprocess-shell-injection      demo/app/s
ervice.py:31
[scan_sast]   HIGH     python-yaml-unsafe-load                demo/app/s
ervice.py:35
[scan_sast]   HIGH     python-subprocess-shell-injection      demo/legac
y/test_service_fixtures.py:7
[scan_sast]   HIGH     python-sqli-string-built-query         demo/legac
y/test_service_fixtures.py:15
[scan_sast]   HIGH     python-eval-dynamic                    demo/legac
y/test_service_fixtures.py:19
EXIT=1
```

Pass	SARIF 2.1.0 with 4 results and 3 rules. Without noise reduction: 11 findings, 5 HIGH, EXIT=1 (versus 4 findings, 0 HIGH, EXIT=0 with it). The extra HIGHs come from test fixtures and safe API forms.

2.3 Inline # nosemgrep cannot hide a finding

The wrapper passes --disable-nosem, so every suppression must live in config/semgrep.yml.

```
printf 'def run(expr):\n    return eval(expr)  # nosemgrep\n' > demo/app/calc.py
./run.sh sast; echo "EXIT=$?"
rm demo/app/calc.py
```

```
┌──(kali㉿kali)-[~/Documents/project_agapi/appsec-takehome]
└─$ cp config/semgrep.yml.bak config/semgrep.yml

┌──(kali㉿kali)-[~/Documents/project_agapi/appsec-takehome]
└─$ printf 'def run(expr):\n    return eval(expr)  # nosemgrep\n' > demo
/app/calc.py

┌──(kali㉿kali)-[~/Documents/project_agapi/appsec-takehome]
└─$ ./run.sh sast; echo "EXIT=$?"
[scan_sast] running semgrep (semgrep/semgrep:1.185.0)
[scan_sast] findings=5 (CRITICAL=0 HIGH=1 MEDIUM=1 LOW=1 INFO=2) MIN_SEV
ERITY=HIGH blocking=1
[scan_sast] findings at/above HIGH:
[scan_sast]   HIGH     python-eval-dynamic                    demo/app/c
alc.py:2
EXIT=1

┌──(kali㉿kali)-[~/Documents/project_agapi/appsec-takehome]
└─$
```

Pass	python-eval-dynamic is reported as HIGH at demo/app/calc.py:2 despite the comment. EXIT=1.

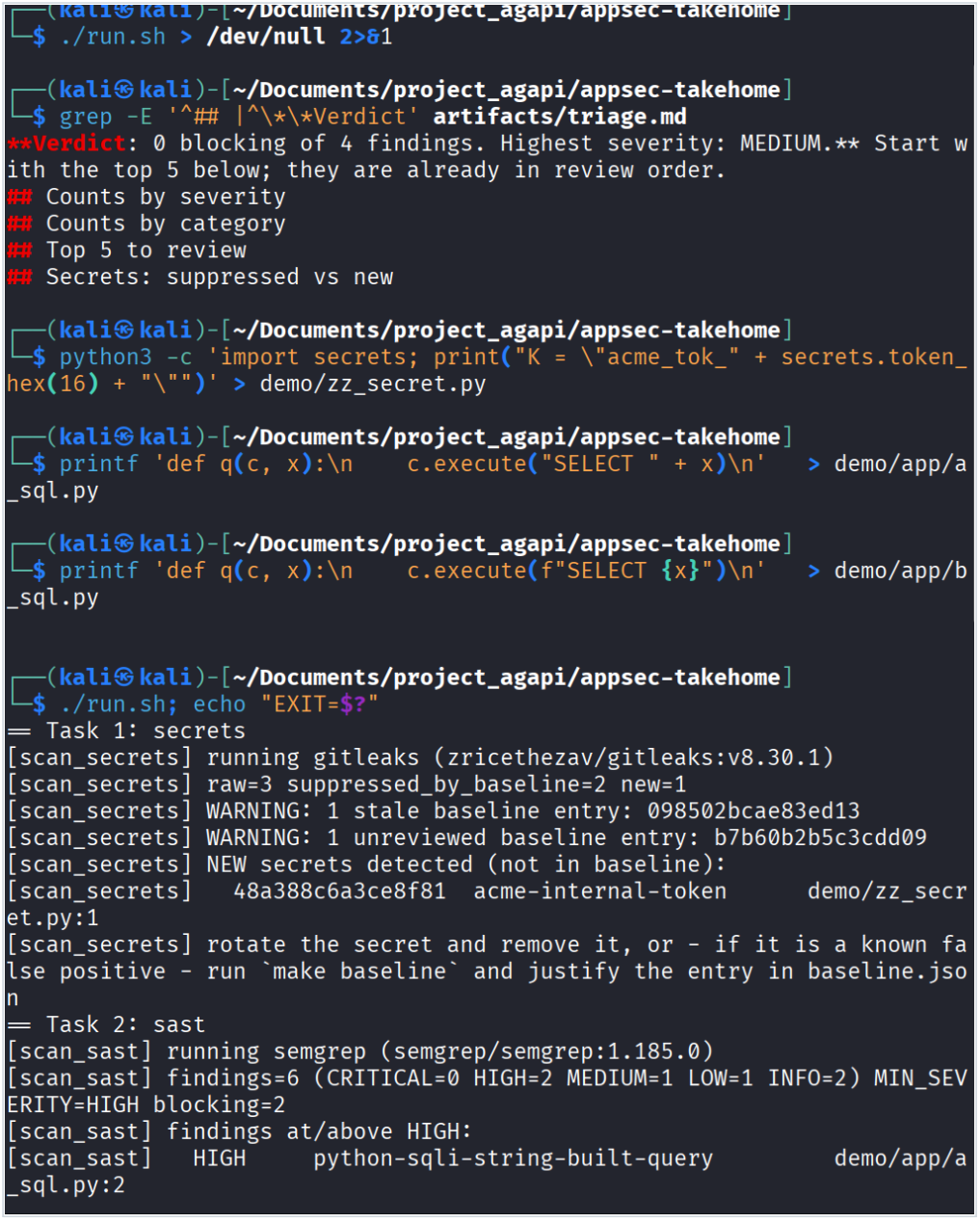
Task 3: Triage report

`triage_report.py` reads the artifacts from Tasks 1 and 2 and writes `artifacts/triage.md`: counts, top 5 (severity desc, then path asc), and suppressed vs new.

3.1 Required sections and stable ordering

Check the four sections, then plant findings whose correct order is known in advance. zz_secret.py sorts last alphabetically, so it can only rank first if severity wins over path.

```
grep -E '^## |^\*\*Verdict' artifacts/triage.md
python3 -c 'import secrets; print("K = \"agapi_tok_" + secrets.token_hex(16) + "\"")' > demo/
    zz_secret.py
printf 'def q(c, x):\n    c.execute("SELECT " + x)\n'  > demo/app/a_sql.py
printf 'def q(c, x):\n    c.execute(f"SELECT {x}")\n'  > demo/app/b_sql.py
./run.sh; echo "EXIT=$?"
sed -n '/## Top 5/,/## Secrets/p' artifacts/triage.md
```



All four sections present; the run exits 1 and still writes the report.

3.1 Required sections and stable ordering (continued)

```
== Task 3: triage report
[triage] wrote /home/kali/Documents/project_agapi/appsec-takehome/artifa
cts/triage.md from sast.json, secrets.filtered.json, secrets.summary.jso
n
== summary: secrets=1 sast=1 triage=0 → exit 1  (report: artifacts/tria
ge.md)
EXIT=1

┌──(kali㉿kali)-[~/Documents/project_agapi/appsec-takehome]
└─$ sed -n '/## Top 5/,/## Secrets/p' artifacts/triage.md
## Top 5 to review

Ordering: severity desc, then file path asc (ties: line, rule).

| # | Severity | Location | Rule | Blocking | Why it matters | Suggested
 fix |
|---:|---|---|---|---|---|---|
| 1 | CRITICAL | `demo/zz_secret.py:1` | `acme-internal-token` | yes | N
ew secret not in baseline (ACME internal service token) | Rotate the cre
dential, then remove it. If it is a known false positive: `make baseline
` and justify the entry. |
| 2 | HIGH | `demo/app/a_sql.py:2` | `python-sqli-string-built-query`<br
>CWE-89: SQL Injection | yes | SQL query built from a string (f-string,
concatenation, % or .format) and passed to execute(). Use a parameterise
d query: execute("... %s", (value,)). | cursor.execute("SELECT ... WHERE
 id = %s", (user_id,)) |
| 3 | HIGH | `demo/app/b_sql.py:2` | `python-sqli-string-built-query`<br
>CWE-89: SQL Injection | yes | SQL query built from a string (f-string,
concatenation, % or .format) and passed to execute(). Use a parameterise
d query: execute("... %s", (value,)). | cursor.execute("SELECT ... WHERE
 id = %s", (user_id,)) |
| 4 | MEDIUM | `demo/app/service.py:11` | `python-requests-tls-verify-di
sabled`<br>CWE-295: Improper Certificate Validation | no | TLS certifica
te verification disabled (verify=False). Traffic can be intercepted. | r
emove verify=False, or verify="/path/to/internal-ca.pem" |
| 5 | LOW | `demo/app/service.py:16` | `python-weak-hash`<br>CWE-328: Us
e of Weak Hash | no | MD5/SHA-1 is broken for security purposes. Use SHA
-256, or pass usedforsecurity=False if this is a non-security checksum.
| hashlib.sha256(data) - or hashlib.md5(data, usedforsecurity=False) |

2 more finding(s) in the full list below.
```

Top 5: CRITICAL zz_secret.py, HIGH a_sql.py, HIGH b_sql.py, MEDIUM, LOW.

Pass	Sections present; ordering matches the rule exactly; the summary line reports secrets=1 sast=1 triage=0 and exit 1.

3.2 Works from either task's output, fails with none

The report must work with Task 1 or Task 2 output alone, and exit 2 when there is nothing to read.

```
make clean; ./run.sh secrets >/dev/null 2>&1; ./run.sh triage; echo "EXIT=$?"
make clean; ./run.sh sast    >/dev/null 2>&1; ./run.sh triage; echo "EXIT=$?"
grep 'Not applicable' artifacts/triage.md
make clean; ./run.sh triage; echo "EXIT=$?"
```



Pass	Secrets only: EXIT=0. SAST only: EXIT=0, with the secrets section marked Not applicable. No input: a clear error and EXIT=2.

Determinism and failure modes

The same repo state must produce byte-identical artifacts, and a broken environment must exit 2, never 0.

4.1 - 4.3 Reruns, missing Docker, make vs ./run.sh

Hash the artifacts across two runs, point DOCKER at a missing binary, and compare make with ./run.sh on a new secret.

```
sha256sum artifacts/* > /tmp/run1.sha; ./run.sh >/dev/null 2>&1
sha256sum -c --quiet /tmp/run1.sha && echo 'identical'
DOCKER=/nonexistent/docker ./run.sh; echo "EXIT=$?"
python3 -c 'import secrets; print("T = \"agapi_tok_" + secrets.token_hex(16) + "\"")' > demo/app/
    leak.py
make secrets; echo "EXIT=$?"      # expect: prints 'exit code: 1', make exits 2
./run.sh secrets; echo "EXIT=$?"  # expect: 1
```

```
┌──(kali㉿kali)-[~/Documents/project_agapi/appsec-takehome]
└─$ sha256sum artifacts/* > /tmp/run1.sha

┌──(kali㉿kali)-[~/Documents/project_agapi/appsec-takehome]
└─$ ./run.sh > /dev/null 2>&1

┌──(kali㉿kali)-[~/Documents/project_agapi/appsec-takehome]
└─$ sha256sum -c --quiet /tmp/run1.sha && echo 'identical'
identical

┌──(kali㉿kali)-[~/Documents/project_agapi/appsec-takehome]
└─$ DOCKER=/nonexistent/docker ./run.sh; echo "EXIT=$?"
== Task 1: secrets
[scan_secrets] ERROR: '/nonexistent/docker' not found - install Docker (
or set GITLEAKS_BIN to a local gitleaks)
== Task 2: sast
[scan_sast] ERROR: '/nonexistent/docker' not found - install Docker (or
set SEMGREP_BIN to a local semgrep)
== Task 3: triage report
[triage] wrote /home/kali/Documents/project_agapi/appsec-takehome/artifa
cts/triage.md from sast.json, secrets.filtered.json, secrets.summary.jso
n
== summary: secrets=2 sast=2 triage=0 → exit 2  (report: artifacts/tria
ge.md)
EXIT=2

┌──(kali㉿kali)-[~/Documents/project_agapi/appsec-takehome]
└─$ python3 -c 'import secrets; print("T = \"agapi_tok_" + secrets.token
_hex(16) + "\"")' > demo/app/leak.py

┌──(kali㉿kali)-[~/Documents/project_agapi/appsec-takehome]
└─$ make secrets; echo "EXIT=$?"
[scan_secrets] running gitleaks (zricethezav/gitleaks:v8.30.1)
[scan_secrets] raw=2 suppressed_by_baseline=2 new=0
[scan_secrets] WARNING: 1 stale baseline entry: 098502bcae83ed13
[scan_secrets] WARNING: 1 unreviewed baseline entry: b7b60b2b5c3cdd09
[scan_secrets] OK: no new secrets
EXIT=0

┌──(kali㉿kali)-[~/Documents/project_agapi/appsec-takehome]
```

Re-run	4.1 identical: pass. 4.2 both scanners report Docker not found and the run exits 2: pass. 4.3 not exercised: the custom rule did not yet match agapi_tok_, so the planted token was not detected (new=0, EXIT=0). Re-run after renaming the rule.